Seminar 2: Additional R Topics

Sean Hellingman ©

Introduction to Statistics (STAT 1200)

shellingman@tru.ca

Fall 2025

THOMPSON RIVERS UNIVERSITY

Topics

2. Introduction
3. R Quarto
4. Vectors in R
5. Importing Data
6. Data Frames
7. Types of Data
8. Exercises and References

Introduction

- We are going to continue to learn about some of the capabilities of R Studio.
- R Studio can do many things that make statistical analysis much easier.
- We will cover R Quarto, vectors, data, and importing datasets.
  - *Most of the key concepts needed to apply the methods we are learning in the lectures.*
  - You can find additional slides on R Studio and data manipulation in R Studio here.

R Quarto

Introduction

- We have already used scripts to save and run our code.
- R Quarto is a way to combine R code and results with written commentary in one document
  - Very useful for writing reports or papers.
  - Technically a different language that is easily integrated into R Studio.
  - Able to facilitate the construction of reproducible reports.
- **This will help with assignments and the project.**

Initialization

- If we want to generate PDF outputs (we will), we need to install tinytex.
- In the **Console** run the following lines:
  - `install.packages("tinytex")`
  - `tinytex::install_tinytex()`
- Once installed we are now able to compile most R Markdown documents.
- *May need to install other packages, I can help with this.*

New R Quarto

- To create a new R Quarto file
  File ⇒ New File ⇒ Quarto Document
  - In the pop up window, give the document a *Title*, add an *Author*, and set PDF as the *Default Output Format*.
  - PDF files are smaller and will be easier to submit assignments with.
- The new Quarto document will contain a header and some example code.
  - It's a good idea to periodically save your file as you work.

Example 1

- Create a new R Quarto file.
- Save this file in your TRU OneDrive.
- Click *Render* to practice converting your file to a PDF.
- *Notice there is a new tab in your console window.*

Anatomy

- R Markdown files are generally composed of three components:
  1. YAML header
  2. Formatted text
  3. Code chunk

YAML Header

- YAML stands for 'YAML Ain't Markup Language'
  - Automatically created when you create a new Quarto file through R studio.
  - Contains the metadata and options for the entire document such as the author name, date, output format, etc.

```yaml
---
title: "Quarto Examples"
author: "Sean Hellingman"
format: pdf
editor: visual
---
```

- Many header options: https://bookdown.org/yihui/rmarkdown/html-document.html

Formatted Text

- The advantage of R Quarto is that one can add formatted text to generate reports.
- The text format is **NOT** *What-You-See-Is-What-You-Get*
  - R Quarto is not exactly like MS Word, the user may notice some differences.
  - May seem difficult at first (we will try to keep things simple at first).
  - Makes formatting much easier than using R and MS Word together.

Comments and Headings

- The # symbol is used for heading in the formatted text
  - # is still used to comment in a *code chunk*.
- Enclose a comment between <!-- and -->
- Comments in the formatted text will not be included in the final rendered document.

Code Chunks

- To include R code into your R markdown document you place your code into a **code chunk**.

- **All code chunks begin and end with three backticks ```**
  - The backtick key is usually found on the same key as tilde (∼).
  - Windows shortcut: Ctrl + Alt + I
  - MacOSX shortcut: Cmd + Option + I

```
```{r}
Any valid R code must go between the backticks (```)
```
```

Example 2

- Using your R Quarto file from Example 1, do the following:
  1. Delete all of the generated example code.
  2. Add a header that reads "Example 2"
  3. In plain text below the header write: *This is where formatted text goes.*
  4. Insert a new code chunk.
  5. In the code chunk, add the following lines of code:
     - `V = c(1,3,4,5,2,6,7)`
     - `V`
     - `4 + 7`
     - `#This is a comment`
  6. Save and render your document.
- What does your output look like?

Vectors in R

Introduction

- Vectors are one of the simplest data structures in R
  - Vectors contain elements of the same type (logical, integer, double, character, complex, or raw).
  - Many R functions are designed to operate on vectors.
- Vectors in R are not assigned as row or column vectors
  - If we want a row or column vector we need to use a matrix.

Creating Vectors `c()`

- The simplest (and most important) way to create vectors in R is with the concatenate function `c()`.
- The command `x = c(-2,0,3)` would produce a vector such that
  - First element is -2.
  - Second element is 0.
  - Third element is 3.
- Can also assign different data types to vectors

Positional Indexing

- To extract elements based on their position we write the position within `[]`
- The first positional index is 1 in R (it is 0 in Python).
- Use `Vector.3[2]` to extract the second element of Vector.3.
- Negative indexing allows us to extract everything *except* the negatively indexed position.
  - Use `Vector.3[-2]` to extract every element of Vector.3 apart from position 2.

Ordering Elements

- Sort the values from lowest to highest value we can use the `sort()` function
  - `Vector.Sorted <- sort(Vector.1)` saves the sorted version of `Vector.1`.
- To reverse the sort, from highest to lowest, we can either include the `decreasing = TRUE` argument when using the `sort()` function or first sort the vector using the `sort()` function and then reverse the sorted vector using the `rev()` function.
  - `Vector.Sorted.Rev <- sort(Vector.1, decreasing = TRUE)`
  - `Vector.Sorted.Rev <- rev(sort(Vector.1))`

Example 3

- Using your R Quarto file from Example 1, do the following:
  1. Add a header below Example 2 that reads "Example 3"
  2. Insert a new code chunk.
  3. In the code chunk, do the following:
     - Create a vector from 1 to 10 and call it `V2`.
     - Extract the fourth element of `V2`.
     - Create the following vector: `V3 = c(3,5,-1,5,7,8,12,-2)`
     - Sort `V3` from smallest to largest.
  4. Save and render your document.
- What does your output look like?

Importing Data

Importing Data

- Most of the datasets you will work with come from outside of R.
- There are example dataset that are built into R packages.
- Import from saved files or spreadsheets (Typically CSV files).
- Can also import data directly from the internet (stock data).

Importing Data from R Packages

- Some R packages have built in datasets that are downloaded when the package is.
- To load a datset from an exising R package:
  1. Load the package in R using `library(package_name)`
  2. load the dataset using `data("data_name")`

Example 4

- Using your R Quarto file from Example 1, do the following:
  1. Add a header below Example 3 that reads "Example 4"
  2. Insert a new code chunk.
  3. In the code chunk, do the following:
     - Load the *MASS* package in R.
       `library(MASS)`
     - Load the *birthwt* dataset from the *MASS* package.
       `data(birthwt)`
     - Spend a bit of time familiarizing yourself with the data.
  4. Save and render your document.
- What does your output look like?

Importing Data from CSV Files

- The `read.csv()` command can read CSV files.
  - `data <- read.csv("filename.csv", header = TRUE)` or `data <- read.csv("filename.csv")` if your CSV file has a header.
  - `data <- read.csv("filename.csv", header = FALSE)` if your CSV file doesn't have a header.
- Note that we did not specify the file location
  - This only works if the CSV file is stored in the **Working Directory**.
  - If the file is not in the Working Directory, need to specify the file location.
  - Or change Working Directory to the file location using `setwd("file path")` first.

Example 5

- Using your R Quarto file from Example 1, do the following:
  1. Download the *Football22.csv* dataset from moodle.
  2. Save the file in an appropriate location to work with.
  3. Add a header below Example 4 that reads "Example 5"
  4. Insert a new code chunk.
  5. In the code chunk, do the following:
     - Import the dataset into your R markdown environment.
     - Spend a bit of time familiarizing yourself with the data.
  6. Save and render your document.
- What does your output look like?

Data Frames in R

Data Frames

- Most commonly used data structure to store data.
- May contain a mixture of different data types.
- Excel worksheet-like, but more powerful.
  - Under the hood, a data frame is a list of equal-length vectors.
- All columns are of equal length.

Creating Data Frames

- Imported data is commonly formatted as a data frame.
- May use `data.frame()` to construct a data frame.
- Can convert a matrix to a data frame using `as.data.frame()`.

Positional Indexing

- Positional indexing in data frames is very similar to matrices.
- Use `DataFrame.1[1,]` to return the first row of `DataFrame.1`.
- Use `DataFrame.1[,4]` to return the 4th column of `DataFrame.1`.
- Use `DataFrame.1[1:4,5]` to return rows 1 to 4 of column 5 of `DataFrame.1`.
- To extract a column (variable) by name: `DataFrame.1$Variable.name`

Example 6

- Using your R Quarto file from Example 1, do the following:
  1. Add a header below Example 5 that reads "Example 6"
  2. Insert a new code chunk.
  3. In the code chunk, do the following:
     - Extract and save the vector of league points in *Football22*.
     - Extract the fifth row of the data frame.
     - Extract rows 1 to 5 of column 3 of the data frame.
  4. Save and render your document.
- What does your output look like?

Types of Data in R

Introduction

- It is very important to understand the different types (classes) of data in R.
- There are different rules and operators that apply to different data types.
  - Can't use + to add two characters together.
- There are *six* basic types of data in R.
  1. Numeric
  2. Integer
  3. Logical
  4. Character
  5. *Complex*
  6. *Raw*

1. Numeric

- Numeric data are numbers that contain a decimal.
- *They can technically be whole numbers also*.
- Example: `Num.Object = 5.25`

2. Integers

- Integers are data that **do not** contain a decimal.
- Example: `Int.Object = 5`

3. Logical

- Logical (Boolean) data take on the value of either `TRUE` or `FALSE`.
- Another type of logical data called `NA` represents missing data.
- Example: `Logical.val = 5 < 4`
  - `Logical.val` will equal `FALSE`

4. Character

- Character data are used to represent strings.
  - Can be defined using: "value" or 'value'
  - Example: `string.variable = 'This is a string'`
- Character strings are something like a word or multiple words.
- Characters can be ordered and turned into *factors*.
  - `as.factor(variable)`

References & Resources

1. Douglas, A., Roos, D., Mancini, F., Couto, A., & Lusseau, D. (2023). *An Introduction to R*

- ADSC1010 Slides
- Data Types
- MASS package